

Codefest 3: Solutions

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

CMAN — DRAM traffic analysis: naive vs. tiled matrix multiply

Setup

$N = 32$, FP32 (4 bytes/element), row-major storage. Tile size $T = 8$.

Task 1: Naive DRAM traffic

For the naive triple loop (ijk order), each output element $C[i][j]$ requires reading N elements from row i of A and N elements from column j of B . Assuming no data reuse (every access goes to DRAM), across all N^2 output elements:

- Accesses to A : N^2 output elements $\times N$ reads each = N^3 element accesses
- Accesses to B : N^2 output elements $\times N$ reads each = N^3 element accesses
- Each element of $B[k][j]$ is accessed $N = 32$ times (once per output row i , for fixed j)

Formula:

$$\begin{aligned} \text{Traffic_naive} &= 2 \times N^3 \times 4 \text{ bytes} \\ &= 2 \times 32^3 \times 4 = 2 \times 32,768 \times 4 = 262,144 \text{ bytes} = 256 \text{ KB} \end{aligned}$$

Arithmetic intensity: FLOPs = $2N^3 = 65,536$; Bytes $\approx 262,144$; AI = $65,536 / 262,144 \approx 0.25$ FLOP/byte.

Task 2: Tiled DRAM traffic ($T = 8$)

With ideal tiling, each element of A and B is loaded from DRAM exactly once. The tile machinery ensures that a $T \times T$ block loaded into shared memory is fully consumed before the next tile is fetched, so the total DRAM traffic equals simply loading the two input matrices once each.

- A : N^2 elements loaded once = $32^2 \times 4 = 4,096$ bytes
- B : N^2 elements loaded once = $32^2 \times 4 = 4,096$ bytes

Formula:

$$\begin{aligned} \text{Traffic_tiled} &= 2 \times N^2 \times 4 \text{ bytes} \\ &= 2 \times 32^2 \times 4 = 2 \times 1,024 \times 4 = 8,192 \text{ bytes} = 8 \text{ KB} \end{aligned}$$

Task 3: Traffic ratio

$$\begin{aligned} \text{Ratio} &= \text{Traffic_naive} / \text{Traffic_tiled} \\ &= 262,144 / 8,192 = 32 = N \end{aligned}$$

Algebraic check:

$$\text{Ratio} = (2N^3 \times 4) / (2N^2 \times 4) = N^3 / N^2 = N$$

One-sentence explanation: The ratio equals N because tiling allows each element of A and B to be loaded from DRAM exactly once (reused N times across all output tiles that need it), reducing total DRAM traffic from $2N^3$ bytes to $2N^2$ bytes, a factor of N .

Task 4: Execution times

Given: DRAM bandwidth = 320 GB/s, compute = 10 TFLOPS. Total FLOPs = $2N^3 = 65,536$.

Naive case:

$$\begin{aligned} t_{\text{memory}} &= 262,144 / (320 \times 10^9) \approx 0.819 \text{ } \mu\text{s} \\ t_{\text{compute}} &= 65,536 / (10 \times 10^{12}) \approx 0.007 \text{ } \mu\text{s} \\ \text{Bottleneck: MEMORY. Execution time} &\approx 0.819 \text{ } \mu\text{s}. \end{aligned}$$

Tiled case:

$$\begin{aligned} t_{\text{memory}} &= 8,192 / (320 \times 10^9) \approx 0.026 \text{ } \mu\text{s} \\ t_{\text{compute}} &= 65,536 / (10 \times 10^{12}) \approx 0.007 \text{ } \mu\text{s} \\ \text{Bottleneck: MEMORY (} t_{\text{memory}}/t_{\text{compute}} &\approx 4\times, \text{ but compute is approaching).} \\ \text{Execution time} &\approx 0.026 \text{ } \mu\text{s}. \end{aligned}$$

Note: the tiled case at $N=32$, $T=8$ is still memory-bound but the margin has shrunk from $\sim 117\times$ to $\sim 4\times$. At larger N the tiled kernel would cross the ridge point and become compute-bound.

CLLM — CUDA GEMM: naive vs. tiled with roofline placement

See GitHub repository for all deliverable files. Reference implementations are provided below.

gemm_naive.cu

One thread per output element. Block size 32×32 . Timing via `cudaEventRecord`. Reports elapsed time and achieved GFLOP/s. Path: `codefest/cf03/cuda/gemm_naive.cu`

gemm_tiled.cu

Shared-memory tiled kernel, tile size $T=8$. Each threadblock loads one $T\times T$ tile of A and one $T\times T$ tile of B into shared memory, accumulates the partial dot product, then advances to the next tile. Two `__syncthreads()` barriers per tile step prevent read-before-write and write-before-read hazards. Path: `codefest/cf03/cuda/gemm_tiled.cu`

gemm_analysis.md (≥ 150 words)

- **Why naive is memory-bound:** Each thread independently loads a full row of A and a full column of B from DRAM for a single output element. B accesses are strided and non-coalesced. Arithmetic intensity is 0.25 FLOP/byte, far below the GPU ridge point.
- **How tiling reduces DRAM traffic:** A $T\times T$ tile is loaded once into shared memory and reused for T dot-product contributions before the next fetch. In the ideal limit each matrix element is loaded exactly once, reducing traffic by a factor of N .
- **Remaining bottleneck at $T=8$:** The threadblock has only 64 threads, limiting warp-level parallelism and the GPU's ability to hide memory latency. A larger tile (e.g., $T=32$) would raise both reuse and occupancy, pushing the kernel closer to the compute ceiling.

COPT — Forward pass of a neural network on the GPU

Network: $\text{Linear}(4\rightarrow 5) \rightarrow \text{ReLU} \rightarrow \text{Linear}(5\rightarrow 1)$, batch size 16. The script detects whether a CUDA GPU is available and exits cleanly if not. On a CUDA-capable machine it prints the GPU name, output shape $[16, 1]$, and the output device. Path: `codefest/cf03/copt/nn_forward_gpu.py`

Expected terminal output (CUDA machine):

```
Device: NVIDIA GeForce RTX 3060 (or similar)
Output shape : [16, 1]
Output device: cuda:0
```